



- Stage 1: NIC & Device Driver
 - Packet is transferred from network interface card to ring buffer
- Stage 2: Kernel Protocol Stack
 - Packet is transferred from ring buffer to a socket receive buffer
- Stage 3: Data Receiving Process
 - Packet is copied from the socket receive buffer to the application

Figure 1: Data receiving process

```

Hackintosh~ # ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc pfifo_fast state DOWN group default qlen 1000
    link/ether 28:02:44:c1:7b:ce brd ff:ff:ff:ff:ff:ff
  
```

Figure 2: `ip addr` output for loopback and `eth0` interface

```

Hackintosh~ # netstat -ta
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address          Foreign Address         State
tcp        0      0 0.0.0.0:localhost:22    *:*                      LISTEN
tcp        0      0 0.0.0.0:ssh:22         *:*                      LISTEN
tcp        0      0 0.0.0.0:localhost:22    *:*                      LISTEN
  
```

Figure 3: `netstat-ta` shows all the `tcp` connections happening in the system

netstat: This utility shows all the network connections happening in your system. It has lots of statistics to show, and can be useful to find bottlenecks or problems.

ethtool: To tune your network, `ethtool` can be your best friend. It is used to display and change the settings of your network interface card.

ethtool eth0: This shows the hardware setting of the interface `eth0`.

ss: This is another helpful tool to see network statistics and works the same as `netstat`.

ss -t: This shows all the network ports currently opened.

Benchmark before tuning

To get maximum or even improved network performance, our goal is to increase the throughput (data transfer rate) and latency of our network's receiving and sending capabilities. But tuning without measurement (effected values) is useless as well as dangerous. So, it is always advisable to benchmark every change you make, because any change that doesn't result in an improvement is pointless and may degrade performance. There are a few benchmarking tools available for networks, of which the following two can be the best choices.

Netperf: This is a perfect tool to measure the different aspects of a network's performance. Its primary focus is on data transfer using either TCP or UDP. It requires a server and a client to run tests. The server should be run by a

```

Hackintosh~ # ethtool eth0
Settings for eth0:
Supported ports: [ 10P MII ]
Supported link modes: 10baseT/Half 10baseT/Full
                  100baseT/Half 100baseT/Full
                  1000baseT/Half 1000baseT/Full
Supported pause frame use: No
Supports auto-negotiation: Yes
Advertised link modes: 10baseT/Half 10baseT/Full
                  100baseT/Half 100baseT/Full
                  1000baseT/Full
Advertised pause frame use: Symmetric Receive-only
Advertised auto-negotiation: Yes
Speed: 1000Mbps
Duplex: Half
Port: MII
PerMCG: 0
Transceiver: internal
Auto-negotiation: on
Supports Wake-on: pumbg
Wake-on: d
Current message level: 0x00000033 (51)
drv probe ifdown ifup
Link detected: no
  
```

Figure 4: The entire Net adapter details shown by `ethtool eth0`

NetId	State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port
n1	UNCONN	0	0	rtm1(gn-damun)/3660	*
n1	UNCONN	0	0	rtm1(gn-damun)/2816	*
n1	UNCONN	0	0	rtm1(ned)/1465	*
n1	UNCONN	0	0	rtm1(NetworkManager)/521	*
n1	UNCONN	0	0	rtm1(sshd)	*
n1	UNCONN	0	0	rtm1(ned)/2249	*
n1	UNCONN	0	0	rtm1(4359429)	*
n1	UNCONN	0	0	rtm1(sshd-damun)/1375	*
n1	UNCONN	0	0	rtm1(sshd-control)/3664	*
n1	UNCONN	0	0	rtm1(sshd-control)/3664	*
n1	UNCONN	0	0	rtm1(gn-damun)/3660	*
n1	UNCONN	0	0	rtm1(sshd-damun)/5556	*
n1	UNCONN	0	0	rtm1(ned)/2249	*
n1	UNCONN	0	0	rtm1(4135825)	*
n1	UNCONN	0	0	rtm1(sshd-damun)/1375	*
n1	UNCONN	0	0	rtm1(ned)/1465	*
n1	UNCONN	768	0	tcpdmg:kernel	*
n1	UNCONN	4092	0	tcpdmg:19219	*
n1	UNCONN	0	0	ac1nuc:kernel	*
n1	UNCONN	0	0	ac1nuc:kernel	*
n1	UNCONN	0	0	ac1nuc:systemd/1	*
n1	UNCONN	0	0	ac1nuc:auditd/396	*
n1	UNCONN	0	0	rtm1(loopback:kernel)	*

Figure 5: `ss output`

daemon called `netserver`, through which the client (testing system) gets connected.

By running both server and client on `localhost`, I got the output shown in Figure 6.

Remember that the `netperf` default runs on Port 12865 and shows results for an IPv4 connection. `Netperf` supports a lot of different types of tests but for simplicity, you can use default scripts located at `/usr/share/doc/netperf/examples`. It is recommended that you read the `netperf` manual for clarity on how to use it.

Ipperf: This tool performs network throughput tests and has to be your main tool for testing. It also needs a server and client, just like `netperf`.

To run `iperf` as the server, use `$iperf -s -p port`, where 'port' can be any random port.

On the client side, to connect to the server, use `$iperf -c server_ip -p server_port`. The result (Figure 8) has been tested on my `localhost` as both server and client, so the output is not appropriate.

Start the tuning now

Before proceeding to tune your system, you must know that every system differs, whether it is in terms of the CPU, memory, architecture or other hardware configuration. So a tunable parameter may or may not enhance the performance